Project Coding – Database schema (PostgreSQL)

Seminar Hall Booking System – full Tables.sql DDL

Full source code – Included the most important code segments only (core schema, routing, JWT auth, request creation, department approval, overlap checks, and key UI logic). Omitted lines are marked with // ... or -- .... Full sources are on GitHub. Repository: https://github.com/sreejithr247/seminar-hall-booking-system.

DB/Tables.sql

```
-- ============================================
-- SEMINAR HALL BOOKING SYSTEM SCHEMA
-- ============================================
-- 1. Custom ENUM Types
CREATE TYPE USER_ROLE AS ENUM ('admin', 'dept_coordinator', 'requester', 'faculty');
CREATE TYPE APPROVAL_STATUS AS ENUM ('pending', 'forwarded', 'approved', 'rejected', 'amended');
CREATE TYPE BOOKING_STATUS AS ENUM ('confirmed', 'cancelled', 'completed', 'no_show');
CREATE TYPE REQUESTER_TYPE AS ENUM ('class', 'club');
-- 2. Core Tables
CREATE TABLE departments (
    dept_id      SERIAL PRIMARY KEY,
    dept_name    VARCHAR(100) NOT NULL UNIQUE,
    dept_code    VARCHAR(10)  UNIQUE,
    created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE classes (
    class_id     SERIAL PRIMARY KEY,
    class_name   VARCHAR(100) NOT NULL,           -- e.g., BCA 2023-26
    dept_id      INTEGER NOT NULL REFERENCES departments(dept_id),
    year         VARCHAR(10),
    UNIQUE(dept_id, class_name)
);

CREATE TABLE clubs (
    club_id      SERIAL PRIMARY KEY,
    club_name    VARCHAR(150) NOT NULL UNIQUE,    -- e.g., Robotics Club
    dept_id      INTEGER REFERENCES departments(dept_id),  -- NULL = college-level
    description  TEXT
);

CREATE TABLE halls (
    hall_id      SERIAL PRIMARY KEY,
    hall_name    VARCHAR(100) NOT NULL,
    capacity     INTEGER NOT NULL CHECK (capacity > 0),
    location     VARCHAR(100),
    facilities   JSONB DEFAULT '{}',             -- {"projector": true, "ac": true}
    is_active    BOOLEAN DEFAULT true
);
-- 3. Users & Requesters (Class or Club representative)
CREATE TABLE users (
    user_id       SERIAL PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE,
    phone         VARCHAR(15),
    role          USER_ROLE NOT NULL,
    dept_id       INTEGER REFERENCES departments(dept_id),
    is_active     BOOLEAN DEFAULT true,
    created_at    TIMESTAMPTZ DEFAULT NOW(),
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);
-- One user can represent only ONE entity: either a Class or a Club
CREATE TABLE requesters (
    requester_id      SERIAL PRIMARY KEY,
    user_id           INTEGER NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    requester_type    REQUESTER_TYPE NOT NULL,
    class_id          INTEGER REFERENCES classes(class_id),
    club_id           INTEGER REFERENCES clubs(club_id),
    created_at        TIMESTAMPTZ DEFAULT NOW(),
    CONSTRAINT valid_requester
        CHECK (
            (requester_type = 'class' AND class_id IS NOT NULL AND club_id IS NULL) OR
            (requester_type = 'club' AND club_id IS NOT NULL AND class_id IS NULL)
        )
);
-- 4. Booking Requests & Workflow
CREATE TABLE requests (
    request_id          SERIAL PRIMARY KEY,
    requester_id        INTEGER NOT NULL REFERENCES requesters(requester_id),
    hall_id             INTEGER NOT NULL REFERENCES halls(hall_id),
    event_title         VARCHAR(200) NOT NULL,
    event_description   TEXT,
    event_date          DATE NOT NULL,
    start_time          TIME NOT NULL,
    end_time            TIME NOT NULL,
    expected_attendees  INTEGER CHECK (expected_attendees > 0),
    purpose             VARCHAR(300),
    dept_status         APPROVAL_STATUS DEFAULT 'pending',
```

```
    dept_approved_by    INTEGER REFERENCES users(user_id),
    dept_approved_at    TIMESTAMPTZ,
    dept_remarks        TEXT,
    admin_status        APPROVAL_STATUS DEFAULT 'pending',
    admin_approved_by   INTEGER REFERENCES users(user_id),
    admin_approved_at   TIMESTAMPTZ,
    admin_remarks       TEXT,
    requested_at        TIMESTAMPTZ DEFAULT NOW(),
    is_cancelled        BOOLEAN DEFAULT false,
    CONSTRAINT valid_time CHECK (end_time > start_time)
);
-- 5. Final Bookings (only after admin approval)
CREATE TABLE bookings (
    booking_id     SERIAL PRIMARY KEY,
    request_id     INTEGER NOT NULL UNIQUE REFERENCES requests(request_id) ON DELETE RESTRICT,
    hall_id        INTEGER NOT NULL REFERENCES halls(hall_id),
    requester_id   INTEGER NOT NULL REFERENCES requesters(requester_id),
    event_title    VARCHAR(200) NOT NULL,
    event_date     DATE NOT NULL,
    start_time     TIME NOT NULL,
    end_time       TIME NOT NULL,
    status         BOOKING_STATUS DEFAULT 'confirmed',
    created_at     TIMESTAMPTZ DEFAULT NOW(),
    completed_at   TIMESTAMPTZ,
    CONSTRAINT valid_booking_time CHECK (end_time > start_time)
);
-- Checks: No other active booking in same hall/date where times overlap
CREATE OR REPLACE FUNCTION check_booking_overlap()
RETURNS TRIGGER AS $$
DECLARE
    overlap_count INTEGER;
BEGIN
    -- Query for overlaps (exclude self for updates, ignore cancelled)
    SELECT COUNT(*)
    INTO overlap_count
    FROM bookings b
    WHERE b.hall_id = NEW.hall_id
      AND b.event_date = NEW.event_date
      AND b.status != 'cancelled'
      AND (
          (NEW.start_time < b.end_time AND NEW.end_time > b.start_time)
      )
      AND b.booking_id != COALESCE(NEW.booking_id, 0);  -- Exclude current row

    IF overlap_count > 0 THEN
        RAISE EXCEPTION 'Booking overlaps with an existing booking on hall % for date %. Time conflict detected.',
NEW.hall_id, NEW.event_date;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- Attach trigger to bookings table
CREATE TRIGGER trig_check_booking_overlap
    BEFORE INSERT OR UPDATE OF start_time, end_time, event_date, hall_id, status
    ON bookings
    FOR EACH ROW
    EXECUTE FUNCTION check_booking_overlap();
-- 6. Audit & Session Management
CREATE TABLE booking_audit_log (
    log_id      SERIAL PRIMARY KEY,
    booking_id  INTEGER REFERENCES bookings(booking_id) ON DELETE SET NULL,
    request_id  INTEGER REFERENCES requests(request_id),
    action      VARCHAR(50) NOT NULL,
    acted_by    INTEGER NOT NULL REFERENCES users(user_id),
    acted_at    TIMESTAMPTZ DEFAULT NOW(),
    remarks     TEXT,
    ip_address  INET
);

CREATE TABLE login_sessions (
    session_id     VARCHAR(255) PRIMARY KEY,
    user_id        INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    ip_address     INET,
    user_agent     TEXT,
    login_at       TIMESTAMPTZ DEFAULT NOW(),
    last_activity  TIMESTAMPTZ DEFAULT NOW(),
    logout_at      TIMESTAMPTZ,
    is_active      BOOLEAN GENERATED ALWAYS AS (logout_at IS NULL) STORED
);

CREATE TABLE notifications (
    notif_id    SERIAL PRIMARY KEY,
    user_id     INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    title       VARCHAR(150),
    message     TEXT NOT NULL,
    notif_type  VARCHAR(30) DEFAULT 'info',
    is_read     BOOLEAN DEFAULT false,
    created_at  TIMESTAMPTZ DEFAULT NOW()
);
-- 7. Indexes for Performance (helps with overlap queries too)
```

```
CREATE INDEX idx_bookings_overlap_check ON bookings (hall_id, event_date, start_time, end_time) WHERE status !=
'cancelled';
CREATE INDEX idx_requests_date_hall ON requests(event_date, hall_id);
CREATE INDEX idx_requests_requester ON requests(requester_id);
CREATE INDEX idx_requests_status ON requests(dept_status, admin_status);
CREATE INDEX idx_login_sessions_user ON login_sessions(user_id) WHERE is_active = true;

-- 8. Trigger: Auto-update last_activity on session updates
CREATE OR REPLACE FUNCTION update_session_activity()
RETURNS TRIGGER AS $$
BEGIN
    NEW.last_activity = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_update_session_activity
    BEFORE UPDATE ON login_sessions
    FOR EACH ROW
    EXECUTE FUNCTION update_session_activity();
```